Ramificaciones en Git

Modelo interno

- Una rama es un puntero móvil de 41 bytes a un *commit*, por lo que crearla o eliminarla es casi instantáneo.
- HEAD es un puntero a la rama actual.
- Cada *commit* referencia a su padre, y así se forma el grafo acíclico dirigido del historial.

Operaciones básicas

- git branch lista, crea o elimina ramas (-d para fusionadas, -D para forzar).
- git switch cambia de rama y git switch -c crea y cambia en un paso. En versiones anteriores se usaba git checkout.

Integración con merge

- **Fast-forward:** si la rama destino es ancestro directa, solo se avanza el puntero.
- **Fusión a tres bandas:** si hay divergencia, Git usa el ancestro común y los dos extremos para crear un *commit* de fusión con dos padres.
- Para resolver conflictos se editan los marcadores <<<<<<<, ======= y >>>>>>>, se ejecuta git add y se confirma. git merge --abort cancela la fusión.

Gestión de ramas

- git branch -v muestra el último *commit* de cada rama.
- --merged y --no-merged indican qué ramas ya se integraron y pueden eliminarse.

Flujos de trabajo

- Ramas de larga duración (p. ej. main, develop) y ramas temáticas de vida corta (una por funcionalidad o corrección).

Ramas remotas

- Las ramas de seguimiento remoto (origin/main) reflejan el último estado conocido del remoto y se actualizan con fetch.
- Una rama local puede configurarse como de seguimiento (*tracking*) de una remota.
- git push origin --delete <rama> elimina una rama remota.

Rebase

- Reaplica los *commits* de una rama sobre otra base y produce un historial lineal.
- Regla: no hacer *rebase* de *commits* ya publicados, porque reescribe el historial y afecta a quienes lo hayan descargado.

Referencia

S. Chacon y B. Straub, *Pro Git*, 2.ª ed. Apress, 2014. [En línea]. Disponible en: https://git-scm.com/book/es/v2